# API de Gerenciamento de Tarefas na Azure

Relatorio Tecnico

## Sumario

Sumario ..... 2
Capitulo 1 — Introducao ..... 5
Capitulo 2 — Objetivos ..... 5
Objetivo geral ..... 5
Objetivos especificos ..... 5
Capitulo 3 — Tecnologias utilizadas ..... 6
Python ..... 6
FastAPI ..... 6
Pydantic ..... 6
SQLAlchemy ..... 6
SQLite ..... 6
Uvicorn ..... 6
Nginx ..... 7
Git e GitHub ..... 7
Capitulo 4 — Arquitetura da solucao ..... 7
Capitulo 5 — Desenvolvimento da API ..... 8
Organizacao do codigo ..... 8
Endpoints implementados ..... 8
Validacao e tratamento de erros ..... 8
Capitulo 6 — Banco de dados ..... 9
O que e o SQLite ..... 9
Por que foi escolhido ..... 9
Como funciona sua persistencia ..... 9
Onde o arquivo e armazenado ..... 9
O que acontece quando a aplicacao reinicia ..... 10
Limitacoes do SQLite ..... 10
Por que e adequado para um projeto academico pequeno ..... 10
Armazenamento local persistente vs. disco temporario vs. armazenamento gerenciado Azure ..... 10
Capitulo 7 — Infraestrutura Azure ..... 11
Computacao em nuvem e modelo IaaS ..... 11
Assinatura Azure (Subscription) ..... 11
Resource Group ..... 11
Virtual Machine ..... 11
SKU ..... 12

Caracteristicas e justificativa da SKU utilizada ..... 12
vCPU e memoria RAM ..... 12
Papel do Ubuntu Server ..... 12
Managed Disk ..... 12
Diferenca entre disco persistente e temporario ..... 12
Virtual Network e Subnet ..... 12
Public IP ..... 13
NSG (Network Security Group) ..... 13
Como a aplicacao recebe requisicoes ..... 13
Como o SQLite armazena os registros ..... 13
Como Nginx e Uvicorn trabalham juntos ..... 13
Configuracoes basicas de seguranca ..... 13
Custos e limitacoes ..... 14
Capitulo 8 — Implantacao ..... 14
Passos efetivamente executados neste trabalho (ambiente local) ..... 14
Passos planejados para a VM Azure (dependem de acao manual, ver `GUIA_AZURE.md`) ..... 14
Capitulo 8-A — Implantacao realizada (dados reais) ..... 15
Ajuste de plano: SKU e regiao ..... 15
Recursos criados e por que cada um existe ..... 15
Camada de software na VM ..... 16
Protecao de escrita e HTTPS ..... 16
Resultados dos testes na URL publica ..... 16
Capitulo 9 — Testes e validacoes ..... 16
Testes automatizados ..... 16
Testes manuais executados (via curl, ambiente local) ..... 17
Testes planejados (dependem do deploy Azure) ..... 17
Capitulo 10 — Evidencias ..... 17
Capitulo 11 — Custos ..... 18
Recursos que podem gerar cobranca ..... 18
Recomendacoes de controle de custo ..... 19
Capitulo 12 — Seguranca e limitacoes ..... 19
Controles implementados ..... 19
Riscos residuais e limitacoes ..... 19
Capitulo 13 — Conclusao ..... 20
Capitulo 14 — Referencias ..... 21

## Capitulo 1 — Introducao

Este trabalho apresenta o desenvolvimento de uma API REST para gerenciamento de tarefas, construida em Python com o framework FastAPI, com persistencia de dados em SQLite, preparada para hospedagem em uma Virtual Machine da Microsoft Azure. O projeto cobre o ciclo completo de uma aplicacao web moderna: modelagem de dados, implementacao de endpoints HTTP, validacao e tratamento de erros, testes automatizados, controle de versao, documentacao tecnica e infraestrutura de implantacao em nuvem.

O cenario escolhido — um gerenciador de tarefas — e deliberadamente simples, para que o foco do trabalho recaia sobre a correta aplicacao dos conceitos de API REST, persistencia e infraestrutura em nuvem, e nao sobre a complexidade do dominio de negocio.

## Capitulo 2 — Objetivos

### Objetivo geral

Desenvolver, testar e documentar uma API REST funcional, com persistencia de dados, pronta para ser hospedada em uma Virtual Machine Linux na Microsoft Azure.

### Objetivos especificos

- Implementar operacoes CRUD completas (criar, listar, consultar, atualizar e excluir tarefas).
- Validar os dados de entrada e tratar erros HTTP de forma consistente.
- Persistir os dados em um banco SQLite, garantindo integridade entre reinicializacoes.
- Gerar documentacao interativa automatica (Swagger/OpenAPI).
- Cobrir a aplicacao com testes automatizados (Pytest/HTTPX).
- Versionar o codigo com Git e publica-lo no GitHub.
- Planejar e documentar a infraestrutura de hospedagem na Azure (Virtual Machine, rede, disco e seguranca).
- Produzir documentacao tecnica e academica completa, com diagramas e evidencias.

## Capitulo 3 — Tecnologias utilizadas

### Python

Linguagem de programacao interpretada, multiparadigma, escolhida pela simplicidade de sintaxe e pelo ecossistema maduro para desenvolvimento web. Utilizada para implementar toda a logica da API.

### FastAPI

Framework web Python para construcao de APIs, baseado em tipagem (type hints) e no padrao ASGI. Escolhido por gerar automaticamente a documentacao interativa (Swagger/OpenAPI), por integrar validacao de dados nativamente via Pydantic, e pela produtividade no desenvolvimento de endpoints REST.

### Pydantic

Biblioteca de validacao de dados baseada em anotacoes de tipo Python. Utilizada para definir e validar os formatos de entrada (`TarefaCreate`, `TarefaUpdate`) e saida (`TarefaOut`) da API, gerando automaticamente respostas de erro HTTP 422 quando os dados nao atendem aos criterios definidos (ex.: titulo obrigatorio, tamanho maximo de campos).

### SQLAlchemy

Biblioteca de mapeamento objeto-relacional (ORM) para Python. Utilizada para definir o modelo de dados (`Tarefa`) como uma classe Python e traduzir as operacoes CRUD em comandos SQL para o SQLite, sem a necessidade de escrever SQL manualmente na maior parte do codigo.

### SQLite

Sistema de banco de dados relacional embutido, que armazena todo o banco em um unico arquivo no disco. Escolhido por nao exigir um servidor de banco de dados separado, por ter configuracao minima e por ser suficiente para o volume de dados e de acessos deste projeto academico. Detalhado no Capitulo 6.

### Uvicorn

Servidor ASGI (Asynchronous Server Gateway Interface) utilizado para executar a aplicacao FastAPI. Na VM Azure, roda como um servico

systemd, escutando apenas em `127.0.0.1:8000`.

### Nginx

Servidor web utilizado como proxy reverso na VM Azure: recebe as requisicoes HTTP/HTTPS publicas (portas 80/443) e as repassa internamente ao Uvicorn, evitando expor o servidor de aplicacao diretamente a internet.

### Git e GitHub

Git e o sistema de controle de versao utilizado para registrar o historico de alteracoes do codigo-fonte. GitHub e a plataforma de hospedagem remota do repositorio, utilizada para publicacao, backup e, potencialmente, para o processo de implantacao na VM (via `git clone`/`git pull`).

## Capitulo 4 — Arquitetura da solucao

A arquitetura e composta por tres camadas principais:

1. **Camada de rede Azure**: IP publico, Network Security Group (NSG) e Virtual Network/Subnet, responsaveis por controlar o trafego que chega a VM.
2. **Camada de execucao na VM**: Nginx (proxy reverso), Uvicorn (servidor ASGI) e a aplicacao FastAPI.
3. **Camada de persistencia**: SQLite, armazenado no disco gerenciado (Managed Disk) da VM, em um diretorio persistente (`/var/lib/api-tarefas`).

O fluxo de uma requisicao e: o cliente faz uma chamada HTTP ao IP publico da VM; o NSG permite a passagem (se a porta estiver liberada para a origem); o Nginx recebe a conexao na porta 80/443 e repassa internamente para o Uvicorn em `127.0.0.1:8000`; o FastAPI valida os dados (Pydantic), executa a operacao no SQLite (via SQLAlchemy) e retorna a resposta HTTP, que percorre o caminho inverso at o cliente.

Ver diagramas completos em `docs/diagramas/`:

- Diagrama 1 — Arquitetura Azure.
- Diagrama 2 — Fluxo de uma requisicao HTTP.
- Diagrama 3 — Modelo entidade-relacionamento.
- Diagrama 4 — Processo de deploy.
- Diagrama 5 — Processo de testes.

## Capitulo 5 — Desenvolvimento da API

### Organizacao do codigo

```
app/
├── __init__.py     # Metadados do pacote
├── config.py       # Configuracoes (variaveis de ambiente)
├── database.py     # Engine, sessao e dependency do SQLAlchemy
├── models.py        # Modelo ORM (tabela "tarefas")
├── schemas.py       # Schemas Pydantic (entrada/saida da API)
├── crud.py          # Operacoes de banco de dados (camada de acesso a dados)
└── main.py          # Endpoints HTTP, ciclo de vida e tratamento de erros
```

Essa separacao segue o principio de responsabilidade unica: `models.py` define a estrutura de persistencia, `schemas.py` define o contrato HTTP, `crud.py` isola o acesso a dados, e `main.py` concentra apenas a camada de apresentacao HTTP (rotas, codigos de status, dependencias).

### Endpoints implementados

| Metodo | Rota | Funcao | Codigos de resposta |
|---|---|---|---|
| **GET** | `/health` | Verifica status da aplicacao e do banco | 200 |
| **GET** | `/tarefas` | Lista tarefas com paginacao (`skip`, `limit`) | 200, 422 |
| **GET** | `/tarefas/{id}` | Consulta uma tarefa especifica | 200, 404 |
| **POST** | `/tarefas` | Cria uma nova tarefa | 201, 422 |
| **PUT** | `/tarefas/{id}` | Atualiza uma tarefa existente | 200, 404, 422 |
| **DELETE** | `/tarefas/{id}` | Exclui uma tarefa existente | 204, 404 |

### Validacao e tratamento de erros

- Campos obrigatorios (`titulo`) e limites de tamanho (`titulo`: 1–200 caracteres; `descricao`: 0–2000 caracteres) sao validados automaticamente pelo Pydantic, retornando HTTP 422 com o detalhe do erro quando violados.
- Recursos inexistentes (`GET`, `PUT`, `DELETE` com `id` invalido) retornam HTTP 404 com uma mensagem de erro legivel (`{"detail":

"Tarefa nao encontrada."}`).

- Erros inesperados de banco de dados ou de execucao sao capturados por manipuladores de excecao globais (`app/main.py`), que registram o erro no log do servidor e retornam HTTP 500 com uma mensagem generica, **sem expor stack traces** ao cliente.
- A paginacao (`skip`, `limit`) e validada com restricoes (`skip >= 0`, `1 <= limit <= 100`), retornando HTTP 422 para valores fora desses limites.

## Capitulo 6 — Banco de dados

### O que e o SQLite

SQLite e um sistema de gerenciamento de banco de dados relacional que nao roda como um processo servidor separado: toda a logica do banco esta embutida na propria aplicacao, e os dados ficam armazenados em um unico arquivo no sistema de arquivos.

### Por que foi escolhido

Para este projeto academico, o SQLite elimina a necessidade de provisionar e configurar um servidor de banco de dados adicional (o que aumentaria custo e complexidade de infraestrutura), ao mesmo tempo em que oferece suporte completo a SQL, transacoes e integridade referencial, suficientes para o volume de dados e de acessos esperado.

### Como funciona sua persistencia

Todas as operacoes de escrita (INSERT, UPDATE, DELETE) sao gravadas diretamente no arquivo `.db` no disco. O projeto habilita o modo **WAL (Write-Ahead Logging)**, que grava alteracoes em um arquivo de log (`-wal`) antes de consolida-las no arquivo principal, reduzindo bloqueios entre leituras e escritas concorrentes.

### Onde o arquivo e armazenado

- **Desenvolvimento local:** `./data/tarefas.db` (dentro do projeto, mas fora do controle de versao — ver `.gitignore`).

- **Producao (VM Azure):** `/var/lib/api-tarefas/tarefas.db`, um diretorio persistente fora do diretorio da aplicacao, com permissões restritas ao usuario de servico `apitarefas`.

### O que acontece quando a aplicacao reinicia

O arquivo `.db` permanece no disco, inalterado. Ao reiniciar, a aplicacao apenas reabre a conexao com o arquivo existente; a instrucao `Base.metadata.create_all()`, executada no ciclo de vida da aplicacao, e segura e **nao recria nem apaga tabelas ja existentes** — ela so cria tabelas que ainda nao existem.

### Limitacoes do SQLite

- Suporta boa performance de leitura, mas escrita concorrente e limitada (um unico escritor por vez no nivel do arquivo).
- Nao possui recursos de replicacao, failover automatico ou escala horizontal nativa.
- Nao e recomendado para aplicacoes com grande volume de escrita simultanea ou multiplos servidores de aplicacao acessando o mesmo banco.

### Por que e adequado para um projeto academico pequeno

O volume de dados e de requisicoes de um trabalho academico de demonstracao e, por natureza, pequeno. O SQLite elimina custo e complexidade de um servidor de banco dedicado, sem comprometer a corretude funcional do CRUD demonstrado.

### Armazenamento local persistente vs. disco temporario vs. armazenamento gerenciado Azure

- **Disco persistente (Managed Disk):** disco virtual anexado à VM, que mantem seus dados entre reinicializacoes e e o local correto para o arquivo SQLite.
- **Disco temporario:** disco local de curta duracao (ex.: 8 GiB na SKU `Standard_B2ats_v2`), usado para arquivos de swap/cache do sistema; **pode ser perdido** em operacoes de manutencao da plataforma — nunca deve armazenar dados de aplicacao.

- **Armazenamento gerenciado Azure (Blob Storage, Azure SQL Database):** servicos de armazenamento/banco de dados completamente gerenciados pela Azure, cobrados e configurados separadamente da VM. **Nao foram utilizados neste projeto** — o SQLite roda inteiramente dentro do disco da VM, e isso nao deve ser confundido com um banco de dados gerenciado em nuvem.

## Capitulo 7 — Infraestrutura Azure

### Computacao em nuvem e modelo IaaS

Computacao em nuvem e a entrega de recursos de TI (processamento, armazenamento, rede) sob demanda, via internet, com cobranca geralmente baseada em uso. O modelo **IaaS (Infrastructure as a Service)** disponibiliza a infraestrutura basica (maquinas virtuais, redes, discos), deixando a cargo do usuario instalar e gerenciar o sistema operacional e as aplicacoes — diferente do PaaS (Platform as a Service), onde a plataforma de execucao e gerenciada pelo provedor.

### Assinatura Azure (Subscription)

E a unidade de cobranca e de controle de acesso da Azure: todos os recursos provisionados (VM, disco, rede, IP) sao associados a uma assinatura, que acumula os custos e define os limites de cota disponiveis.

### Resource Group

E um container logico que agrupa recursos relacionados (VM, disco, IP, rede) para facilitar sua gestao conjunta — incluindo a exclusao de todos os recursos do projeto em uma unica operacao ao final do trabalho.

### Virtual Machine

E um servidor virtualizado, hospedado na infraestrutura da Azure, que executa um sistema operacional completo (neste projeto, Ubuntu Server LTS) sob o controle do usuario.

### SKU

E a identificacao do "tamanho" de uma VM — a combinacao especifica de vCPUs, memoria RAM, capacidade de disco temporario e rede oferecida.

### Caracteristicas e justificativa da SKU utilizada

`Standard_B2ats_v2`: 2 vCPU (AMD), 1 GiB de RAM, do tipo "burstable" (acumula creditos de CPU para absorver picos breves de uso). A SKU inicialmente planejada, `Standard_B1s`, **nao esta disponivel** na assinatura Azure for Students (restricao de SKU e politica de regioes permitidas); a B2ats_v2 foi a de menor custo permitida, com cota, na regiao North Central US. Detalhes em `docs/DEPLOY_AZURE_REAL.md`.

### vCPU e memoria RAM

**vCPU** (virtual CPU) e uma unidade de processamento virtualizada, alocada a VM a partir do hardware fisico do datacenter. **Memoria RAM** e a memoria volatil disponivel para a execucao de processos — na `Standard_B2ats_v2`, apenas 1 GiB (mitigado com 1 GiB de swap), suficiente para rodar o Uvicorn, o Nginx e o SQLite nesta escala de projeto, mas um limite a se observar caso o trafego aumente.

### Papel do Ubuntu Server

Sistema operacional Linux de codigo aberto, amplamente utilizado em servidores, escolhido pela estabilidade, pelo suporte de longo prazo (LTS) e pela compatibilidade nativa com Python, Nginx e systemd.

### Managed Disk

Disco virtual gerenciado pela Azure, anexado a VM, que armazena o sistema operacional e os dados persistentes da aplicacao (incluindo o SQLite). "Gerenciado" significa que a Azure cuida da infraestrutura de armazenamento subjacente (redundancia, disponibilidade), sem que o usuario precise gerir discos fisicos.

### Diferenca entre disco persistente e temporario

Ver explicacao detalhada no Capitulo 6.

### Virtual Network e Subnet

A **Virtual Network (VNet)** e uma rede privada isolada dentro da

Azure, onde os recursos (como a VM) recebem endereços IP internos. A **Subnet** e uma subdivisao dessa rede, usada para organizar e aplicar politicas de seguranca a grupos de recursos.

### Public IP

Endereco IP acessivel pela internet publica, associado a VM (via sua interface de rede), permitindo que clientes externos se conectem a aplicacao hospedada.

### NSG (Network Security Group)

Conjunto de regras de firewall de rede que controla quais portas e origens podem enviar trafego para a VM (ou recebe-lo dela). Atua exclusivamente no controle de trafego — **nao executa nenhuma logica de aplicacao**.

### Como a aplicacao recebe requisicoes

Uma requisicao HTTP externa chega ao IP publico da VM; se a porta for permitida pelo NSG, ela chega a interface de rede da VM; o Nginx (que escuta nas portas 80/443) recebe a conexao e a repassa, via proxy reverso, ao Uvicorn (`127.0.0.1:8000`), que executa a aplicacao FastAPI.

### Como o SQLite armazena os registros

Ver Capitulo 6.

### Como Nginx e Uvicorn trabalham juntos

O Nginx atua como a camada publica, responsavel por TLS (quando configurado), buffering de conexoes e isolamento do servidor de aplicacao. O Uvicorn executa o codigo Python/FastAPI e nunca e exposto diretamente a internet — apenas escuta em `127.0.0.1`, acessivel somente pelo proprio Nginx, na mesma maquina.

### Configuracoes basicas de seguranca

- SSH (porta 22) restrito ao IP do administrador via NSG.
- Porta 8000 (Uvicorn) nunca liberada no NSG.
- Usuario de servico `apitarefas` sem privilegios administrativos e sem shell de login.

- Protecoes do sandbox do systemd (`NoNewPrivileges`, `ProtectSystem`, `ProtectHome`).
- Nenhum segredo ou senha armazenado no repositorio Git.

### Custos e limitacoes

Ver Capitulo 11.

## Capitulo 8 — Implantacao

Esta secao descreve o procedimento de implantacao planejado e os passos efetivamente executados neste trabalho.

### Passos efetivamente executados neste trabalho (ambiente local)

1. Codigo da API implementado (`app/`) e versionado localmente.
2. Ambiente virtual Python criado e dependencias instaladas (`requirements.txt`, `requirements-dev.txt`).
3. Aplicacao executada localmente com `uvicorn app.main:app`.
4. Banco de dados SQLite criado automaticamente na primeira execucao.
5. Endpoints testados manualmente via `curl` (POST, GET, PUT, DELETE, health check) — ver evidencias no Capitulo 10.
6. Persistencia validada entre reinicializacoes do processo local.
7. Suite de testes automatizados executada com sucesso (Capitulo 9).

### Passos planejados para a VM Azure (dependem de acao manual, ver `GUIA_AZURE.md`)

1. Provisionamento do Resource Group, Virtual Network, NSG, Public IP e Virtual Machine (`Standard_B2ats_v2`, Ubuntu Server LTS) no portal Azure.
2. Conexao via SSH.
3. Execucao do script `deploy/install.sh`, que automatiza: atualizacao de pacotes, instalacao de dependencias de sistema, criacao do usuario de servico, criacao dos diretorios de aplicacao e de dados, clonagem do repositorio, criacao do ambiente virtual, instalacao das dependencias Python, configuracao do `.env`, inicializacao do banco, configuracao do systemd e do Nginx.
4. Validacao do health check local e, em seguida, do acesso externo

pelo IP publico.

**Classificacao:** os passos da secao "ambiente local" estao **implementados e verificados**; os passos da secao "VM Azure" estao **planejados e automatizados via script**, mas **pendentes de execucao real**, que depende da autorizacao de custo e da acao manual do aluno no portal Azure (ver Capitulo 13 e secao "I" da resposta final).

## Capitulo 8-A — Implantacao realizada (dados reais)

Esta secao registra o que **de fato** foi criado e testado em 2026-11-02.

O detalhamento completo esta em `docs/DEPLOY_AZURE_REAL.md`.

### Ajuste de plano: SKU e regiao

O plano inicial previa `Standard_B1s`. Ao consultar a assinatura Azure for Students, verificou-se que a B1s nao esta disponivel e que uma politica de "regioes permitidas" limita a criacao de recursos a `southafricanorth`, `northcentralus`, `brazilsouth`, `eastus2` e `mexicocentral`. As VMs liberadas em Brazil South tinham cota zero. A opcao de menor custo com cota foi a `Standard_B2ats_v2` em `northcentralus`.

### Recursos criados e por que cada um existe

| Recurso | Valor real | Por que foi escolhido / como funciona |
|---|---|---|
| **Resource Group `GS_Cloud_API`** | North Central US | Agrupa todos os recursos; apagar o grupo remove tudo de uma vez e encerra a cobranca. |
| **Virtual Network `GS_Cloud_API_VNet` + sub-rede `GS_Cloud_API_Subnet`** | 10.10.0.0/16 e 10.10.1.0/24 | Rede privada isolada onde a VM recebe o IP 10.10.1.4. |
| **Network Security Group `GS_Cloud_API_NSG`** | SSH (22) so do IP do administrador; HTTP (80) publico; demais portas negadas | Firewall da Azure na frente da VM; a porta 8000 do Uvicorn nao e exposta. |
| **IP publico `GS_Cloud_API_IP`** | Standard, estatico, 20.80.10.214 | Endereco fixo para acessar a API pela internet; SKU Standard e fechado por padrao e exige NSG. |
| **VM `GS-Cloud-API-VM`** | Standard_B2ats_v2, 2 vCPU, 1 GiB RAM, Ubuntu 24.04.5 LTS | Menor VM permitida; "burstable" acumula creditos de CPU para picos. Swap de 1 GiB compensa a pouca RAM. |
| **Managed Disk `GS_Cloud_API_Disk`** | Standard SSD LRS, 32 GiB | Disco persistente: o SQLite sobrevive a reinicios da VM. LRS mantem 3 copias no mesmo datacenter. |

| | | |
|---|---|---|
| **Autenticacao SSH** | Chave ed25519, sem senha | Resiste a ataques de forca bruta por senha; a chave privada nunca sai do computador do administrador. |

### Camada de software na VM

O `install.sh` instalou Python 3, o ambiente virtual, as dependencias, o Nginx e o servico systemd. O fluxo de uma requisicao e: internet -> IP publico -> NSG (porta 80) -> Nginx -> Uvicorn/FastAPI (127.0.0.1:8000) -> SQLite (`/var/lib/api-tarefas/tarefas.db`, modo WAL, permissao 600).

O systemd reinicia a API automaticamente se ela falhar.

### Protecao de escrita e HTTPS

A API nao tem autenticacao. Por isso o Nginx libera publicamente apenas leitura (GET/HEAD/OPTIONS); POST/PUT/PATCH/DELETE exigem IP autorizado (retornam 403 para os demais). Nao ha HTTPS, pois nao ha dominio proprio (nao existe certificado valido para um IP); a conexao e HTTP e **nao e segura**. Devem ser usados apenas dados ficticios.

### Resultados dos testes na URL publica

Script `scripts/testar_producao.sh` (saida em `docs/evidencias/testes-producao.txt`): **17 aprovados, 0 falhas**.

| Verificacao | Resultado |
|---|---|
| **GET /health** | 200 |
| **GET /docs (Swagger) e /openapi.json** | 200 |
| **POST /tarefas** | 201 |
| **GET /tarefas e GET /tarefas/{id}** | 200 |
| **PUT /tarefas/{id}** | 200 |
| **POST com titulo vazio** | 422 |
| **GET id inexistente** | 404 |
| **Reinicio do servico + nova consulta** | registro preservado (persistencia) |
| **DELETE /tarefas/{id} e nova consulta** | 204 e 404 |
| **Porta 8000 pela internet** | sem resposta (bloqueada) |
| **SQLite** | journal_mode=wal, integrity_check=ok |
| **Escrita de IP nao autorizado** | 403 |

## Capitulo 9 — Testes e validacoes

### Testes automatizados

Suite implementada em `tests/test_api.py` com `tests/conftest.py` configurando um banco SQLite isolado (arquivo temporario, exclusivo da

sessao de testes).

**Comando executado:** `pytest`

**Resultado real obtido:**

```
============================== 19 passed, 1 warning in 0.57s
==============================
```

Os 19 testes cobrem: health check, cadastro valido, validacao de cadastro invalido (titulo ausente/vazio/excedendo tamanho), listagem vazia e com registros, paginacao (incluindo parametros invalidos), consulta de ID existente e inexistente, atualizacao valida e invalida, atualizacao e exclusao de ID inexistente, exclusao valida, persistencia apos reabrir a conexao com o SQLite, e consistencia dos schemas JSON de resposta.

### Testes manuais executados (via curl, ambiente local)

Durante o desenvolvimento, foram executados manualmente, contra o servidor local (`http://127.0.0.1:8000`):

| Operacao | Resultado obtido |
|---|---|
| **`GET /health`** | HTTP 200, `{"status":"ok","database":"ok", ...}` |
| **`POST /tarefas` (dados validos)** | HTTP 201, corpo com `id` gerado |
| **`GET /tarefas`** | HTTP 200, lista contendo a tarefa criada |
| **`GET /tarefas/{id}`** | HTTP 200, dados consistentes com o cadastro |
| **`PUT /tarefas/{id}`** | HTTP 200, `atualizada_em` maior que `criada_em` |
| **`GET /tarefas/99999` (inexistente)** | HTTP 404, `{"detail":"Tarefa nao encontrada."}` |
| **`DELETE /tarefas/{id}`** | HTTP 204, corpo vazio |
| **`GET /tarefas/{id}` apos DELETE** | HTTP 404 |
| **Reinicio do processo Uvicorn** | Tarefa cadastrada antes do reinicio continuou presente apos reiniciar |
| **`GET /docs` e `GET /openapi.json`** | HTTP 200, todos os 3 caminhos documentados (`/health`, `/tarefas`, `/tarefas/{tarefa_id}`) |

### Testes planejados (dependem do deploy Azure)

Os testes do roteiro `GUIA_TESTES.md` referentes a G (acesso externo), H (systemd/logs na VM) e I (reinicializacao da VM) **nao foram executados** neste momento, pois dependem da VM Azure estar provisionada — ver Capitulo 13.

## Capitulo 10 — Evidencias

As capturas de tela reais devem ser inseridas conforme o roteiro

completo em `docs/GUIA_PRINTS.md`. Abaixo, os marcadores para as

evidencias deste relatorio:

`[INSERIR PRINT REAL — FIGURA 01: ESTRUTURA DE PASTAS NO VS CODE]`

`[INSERIR PRINT REAL — FIGURA 04: SWAGGER COM TODOS OS ENDPOINTS]`

`[INSERIR PRINT REAL — FIGURA 05: REQUISICAO POST (HTTP 201)]`

`[INSERIR PRINT REAL — FIGURA 06: REQUISICAO GET (HTTP 200)]`

`[INSERIR PRINT REAL — FIGURA 07: REQUISICAO PUT (HTTP 200)]`

`[INSERIR PRINT REAL — FIGURA 08: REQUISICAO DELETE (HTTP 204)]`

`[INSERIR PRINT REAL — FIGURA 09: SQLITE COM REGISTROS PERSISTIDOS]`

`[INSERIR PRINT REAL — FIGURA 10: PYTEST — 19 TESTES APROVADOS]`

`[INSERIR PRINT REAL — FIGURA 11: REPOSITORIO NO GITHUB]`

`[INSERIR PRINT REAL — FIGURA 12: RESOURCE GROUP NA AZURE]`

`[INSERIR PRINT REAL — FIGURA 13: VIRTUAL MACHINE NA AZURE]`

`[INSERIR PRINT REAL — FIGURA 14: SKU STANDARD_B1S]`

`[INSERIR PRINT REAL — FIGURA 15: MANAGED DISK]`

`[INSERIR PRINT REAL — FIGURA 16: REGRAS NSG]`

`[INSERIR PRINT REAL — FIGURA 17: TERMINAL SSH CONECTADO]`

`[INSERIR PRINT REAL — FIGURA 18: SERVICO SYSTEMD ATIVO]`

`[INSERIR PRINT REAL — FIGURA 19: NGINX ATIVO]`

`[INSERIR PRINT REAL — FIGURA 20: API PUBLICA ACESSIVEL]`

`[INSERIR PRINT REAL — FIGURA 21: PERSISTENCIA APOS REINICIALIZACAO DA VM]`

## Capitulo 11 — Custos

### Recursos que podem gerar cobranca

| Recurso | Pode gerar custo? | Observacao |
|---|---|---|
| **Virtual Machine `Standard_B2ats_v2`** | Sim (compute) | US$ 0,0094/h ≈ US$ 6,86/mes (pay-as-you-go, North Central US, preco de tabela) |
| **Managed Disk (Standard SSD)** | Sim | ~US$ 0,01533/GiB/mes |
| **Public IP (SKU Standard)** | Sim, mesmo com a VM parada | ~US$ 0,005/h ≈ US$ 3,65/mes |
| **Trafego de saida (egress)** | Sim, acima da cota gratuita | Geralmente os primeiros ~100 GB/mes sao gratuitos |
| **NSG, VNet, Subnet** | Nao (sem custo direto) | — |

**Estimativa total aproximada:** ~US$ 13/mes em pay-as-you-go com a VM ligada 24 h (VM ~6,86 + disco ~2,64 + IP ~3,65), consumidos dos creditos do Azure for Students. Desalocar a VM zera o compute. **Estes valores devem ser reconfirmados na Calculadora de Precos oficial da Azure antes de qualquer provisionamento**, pois variam por regiao e mudam ao longo do tempo.

Fontes: ver Capitulo 14 (Referencias) e `docs/GUIA_AZURE.md`.

### Recomendacoes de controle de custo

- Configurar um alerta de orcamento em "Cost Management + Billing" (nao bloqueia gastos automaticamente, apenas notifica).
- Excluir o Resource Group completo ao final do trabalho academico, eliminando todos os recursos e sua cobranca associada.
- Evitar criar discos maiores que o necessario (um disco de SO padrao ja e suficiente para este projeto).

## Capitulo 12 — Seguranca e limitacoes

### Controles implementados

- Validacao de entrada via Pydantic (tamanho e obrigatoriedade de campos).
- Tratamento de erros sem exposicao de stack traces (HTTP 500 genericos, com detalhes registrados apenas no log do servidor).
- Planejamento de rede restritiva: SSH limitado ao IP do administrador; Uvicorn nunca exposto publicamente, apenas via Nginx.
- Usuario de servico sem privilegios (`apitarefas`), com protecoes de sandbox do systemd.
- Ausencia de segredos no repositorio (`.env`, chaves privadas e bancos reais excluidos via `.gitignore`).

### Riscos residuais e limitacoes

- **Sem autenticacao de usuarios:** qualquer cliente com acesso a rede

publica pode executar qualquer operacao (incluindo escrita) na API. Para um ambiente academico, essa decisao foi deliberada, mas e uma limitacao real.

- **Medida simples proposta** (nao obrigatoria para este escopo): adicionar uma chave de API fixa via cabecalho HTTP (`X-API-Key`), validada por um middleware simples, para reduzir escrita anonima irrestrita. Essa medida nao substitui um sistema de autenticacao real (sem gestao de usuarios, sem expiracao, sem revogacao granular) e deve ser tratada apenas como uma barreira adicional minima, nao como seguranca robusta.
- **SQLite nao e adequado para alta concorrencia de escrita** — uma limitacao arquitetural reconhecida, aceitavel neste escopo academico.
- **HTTPS valido depende de dominio proprio** — sem ele, a demonstracao publica ocorre apenas via HTTP.
- **Sem rate limiting** — a API nao limita a quantidade de requisicoes por cliente, o que a torna vulneravel a abuso de uso em um cenario real de producao (fora do escopo deste trabalho).

## Capitulo 13 — Conclusao

Este trabalho implementou e validou localmente uma API REST completa para gerenciamento de tarefas, com persistencia em SQLite, cobertura de testes automatizados (19 testes aprovados) e documentacao tecnica abrangente. Foram também preparados — mas nao executados, pois dependem de autorizacao de custo e acao manual na conta Azure do aluno — os scripts e guias para a implantacao completa em uma Virtual Machine Ubuntu na Microsoft Azure, incluindo configuracao de rede, systemd e Nginx.

**O que foi efetivamente implementado e demonstrado:**

- Codigo completo e funcional da API (CRUD, validacoes, tratamento de erros, documentacao Swagger).

- Testes automatizados executados com sucesso.
- Testes manuais executados contra o servidor local, com evidencias de codigos HTTP corretos.
- Scripts de infraestrutura (Nginx, systemd, instalacao, deploy, backup) escritos e revisados, prontos para uso.
- Documentacao tecnica, diagramas e guias completos.

**O que permanece como planejamento, pendente de acao do aluno:**

- Provisionamento efetivo dos recursos Azure.
- Publicacao do repositorio no GitHub (requer autenticacao do aluno).
- Execucao do deploy na VM e validacao de acesso externo.
- Captura das evidencias que dependem do portal Azure e do GitHub.

## Capitulo 14 — Referencias


- FastAPI — Documentacao oficial: https://fastapi.tiangolo.com/
- Pydantic — Documentacao oficial: https://docs.pydantic.dev/
- SQLAlchemy — Documentacao oficial: https://docs.sqlalchemy.org/
- SQLite — Documentacao oficial: https://www.sqlite.org/docs.html
- Uvicorn — Documentacao oficial: https://www.uvicorn.org/
- Nginx — Documentacao oficial: https://nginx.org/en/docs/
- Pytest — Documentacao oficial: https://docs.pytest.org/
- HTTPX — Documentacao oficial: https://www.python-httpx.org/
- Git — Documentacao oficial: https://git-scm.com/doc
- GitHub Docs: https://docs.github.com/
- Azure Virtual Machines — Bv1-series: https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/general-purpose/bv1-series
- Azure Virtual Machines pricing: https://azure.microsoft.com/pricing/details/virtual-machines/linux/
- Azure Managed Disks pricing: https://azure.microsoft.com/pricing/details/managed-disks/
- Azure Public IP Addresses pricing: https://azure.microsoft.com/pricing/details/ip-addresses/
- Azure Free Account: https://azure.microsoft.com/pricing/purchase-options/azure-account/
- Azure Network Security Groups — Microsoft Learn: https://learn.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview
- Let's Encrypt (Certbot): https://certbot.eff.org/
- systemd — Documentacao (man pages): https://www.freedesktop.org/software/systemd/man/